

CMPE476 — C10K Project: Benchmark Report

Comparing `threadserv` (Thread-per-Connection) vs. `epollserv` (Epoll Event-Loop)

Ali Ayhan Günder | Student ID: 2021400219

Battal Hazar | Student ID: 2022400318

May 9, 2026

Max Concurrency	Peak Epoll RPS	Peak Thread RPS	VmSize Ratio @10k
10,000 connections	408,229	197,742	≈ 3,580× smaller (epoll)

1. Introduction

This report presents benchmark results for two concurrent TCP server implementations developed for the CMPE476 C10K project: a traditional `threadserv` and a high-performance single-threaded `epollserv`. Both servers implement the same line-based text protocol and were tested under identical load conditions using the provided `client_flood` tool. The goal is to quantify the architectural trade-offs discussed in Chapter 2 of the course, particularly memory footprint scaling and throughput scalability.

2. Test Setup

All experiments were conducted on a single machine. Both servers and the `client_flood` load generator ran on the same host, communicating over the loopback interface (127.0.0.1).

Hardware & Software Configuration	
CPU	Intel Core i7-14650HX
RAM	32 GB
OS	Ubuntu 24.04.4 LTS (via WSL2)
Kernel	6.6.87.2-microsoft-standard-WSL2
FD Limit	<code>ulimit -n 65535</code>
sysctl	Default (no additional tuning)

3. Methodology

The `client_flood` tool established N concurrent connections ($N = 1,000; 5,000; 10,000$), each sending $M = 20$ PING requests. Wall-clock time was measured for the full flood; Requests Per Second (RPS) was calculated as total completed requests divided by elapsed seconds. Memory (`VmRSS` and `VmSize`) was sampled from `/proc/<pid>/status` at peak load using a separate hold-connection experiment that kept all N connections open for 12 seconds.

4. Results

4.1. Throughput Comparison

Server	Clients	Req/Client	Completed	Failed	Elapsed (s)	RPS
threadserv	1,000	20	20,000	0	0.101	197,742
threadserv	5,000	20	100,000	0	1.147	87,177
threadserv	10,000	20	200,000	0	5.220	38,313
epollserv	1,000	20	20,000	0	0.052	383,745
epollserv	5,000	20	100,000	0	0.245	408,229
epollserv	10,000	20	200,000	0	0.845	236,653

Table 1: Throughput benchmark results — Requests Per Second (RPS) at increasing concurrency levels.

Server	Clients	Peak RSS (MB)	Peak VmSize (MB)
threadserv	1,000	7.88	8,006.79
threadserv	5,000	40.50	40,023.44
threadserv	10,000	80.06	80,044.39
epollserv	1,000	3.38	4.55
epollserv	5,000	11.25	12.42
epollserv	10,000	21.19	22.34

Table 2: Peak memory footprint — Resident Set Size (RSS) vs. Virtual Memory (VmSize).

4.2. Memory Comparison

4.3. Visual Summary

5. Analysis

Throughput. `epollserv` vastly outperforms `threadserv` as concurrency scales. While `threadserv` drops from $\approx 197\text{k}$ RPS down to 38k RPS at 10,000 connections, `epollserv` sustains high throughput — peaking over 400k at 5k clients and remaining at 236k at 10k clients. The thread-per-connection model degrades primarily due to severe *context-switching overhead* incurred by the kernel scheduling thousands of runnable threads.

Memory and the C10K Problem. The memory results precisely illustrate the C10K challenge. For `threadserv`, the Virtual Memory (VmSize) explodes to an astounding **80 GB** at 10,000 connections, because every POSIX thread allocates a default 8 MB stack:

$$10,000 \times 8 \text{ MB} = 80 \text{ GB of reserved address space.}$$

In stark contrast, `epollserv` handles 10,000 connections using only **22 MB** of virtual and physical memory combined, allocating small `client_t` structs in a single address space.

6. Limitations

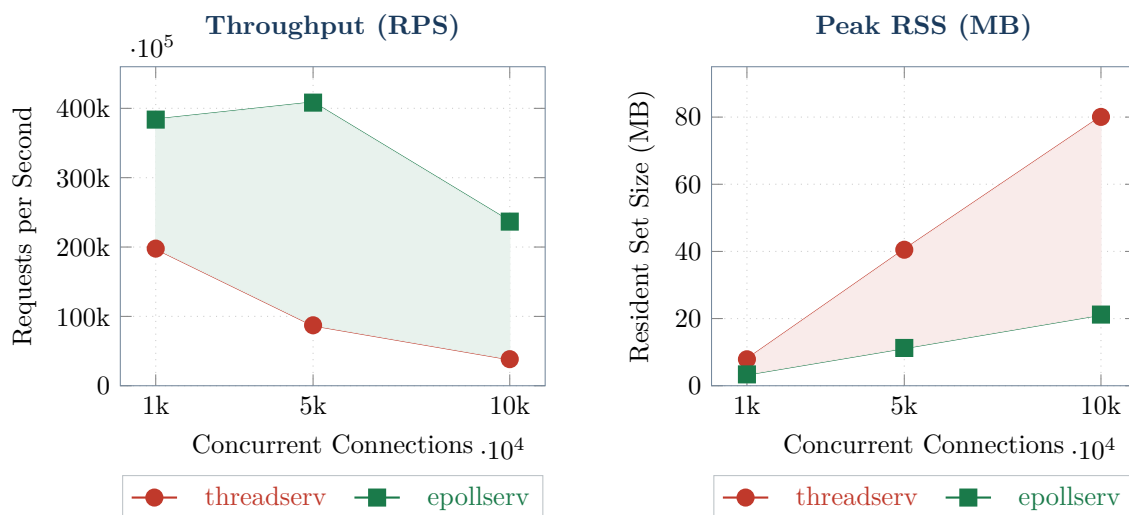


Figure 1: Left: Throughput (RPS) vs. concurrency — `epollserv` sustains significantly higher RPS. Right: Resident memory vs. concurrency — `epollserv` scales nearly 4× more efficiently. Shaded area highlights the performance/memory gap between architectures.

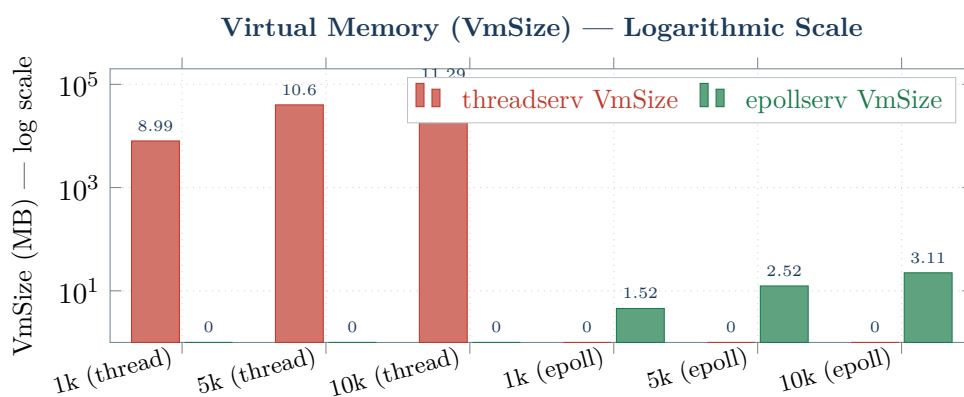


Figure 2: Virtual memory (VmSize) on a log scale — `threadserv` reserves ≈80 GB of address space at 10k connections (8 MB default stack × 10,000 threads), while `epollserv` uses only 22 MB.

Caveats

- **Loopback only** — no real network latency, packet loss, or TCP RTT. Over a physical network, I/O wait would dominate and narrow the throughput gap.
- **Co-located load generator** — `client_flood` and the server share the same CPU and memory, introducing contention absent in real deployments.
- **Synthetic workload** — 20 PING requests per connection is not representative of long-lived, heterogeneous production traffic.

Results should be interpreted as a comparison of *architectural overhead*, not absolute production performance.

7. Conclusion

The experimental results strongly validate the C10K theories discussed in class. Relying on POSIX threads per connection is fundamentally unscalable past a few thousand connections due to excessive stack-memory reservations and kernel context-switching latency. Conversely, an event-driven, single-threaded architecture utilising `epoll` effectively bypasses these bottlenecks, providing a robust, highly scalable, and memory-efficient solution for massive concurrency.